CS 441
Fall 2026
Program 2

Program 2 – The Recursive Descent Parser

We have a program that will evaluate Racket S-expressions. Now we need a way to turn source code into those expressions. Once we have those, our interpreter will be mostly finished.

The process of turning source code into a tree-structure representing the code is *parsing*. Recall that we begin by scanning the code, to tokenize it—to recognize what part of the grammar this is—a number, a punctuation mark, an identifier, a keyword, and so on. Once we've converted our source code into a list of tokens, we can then begin parsing—determining whether the tokens follow the rules of the grammar, and build up the parse tree, the S-expressions representing our program.

The grammar of this language has a few features you may not be familiar with:

- Assignment is handled in the original ALGOL style. Rather than `x := 5`, an assignment is done with the value on the left: `5 => x`.
- The language is typed. The keyword 'decl' indicates a variable declaration, and the type must be specified. Types are: int, real, boolean, string. Example: `decl x, y as real;`
- The language is block-oriented. Blocks of code can nest; inner blocks can access variables declared in outer enclosing blocks. All variables used in a block must be declared at the top of the block.
- This language uses 3-value logic. There are three possible boolean values: "yes", "no", and "maybe".
- Strings are indicated by enclosing them with the backtick character, like this: `` `Hello` => greeting ``
- Strings can only be concatenated using the ~ operator.
- The grammar does not specify comments in code. Here's how to handle them:
    - Introduce comments with two hyphens, like this:
      `` `Hello` => greeting;  -- Greet user with 'Hello' ``
    - During the initial scanning phase, if a double-hyphen is encountered, then scan forward until a newline or end of file is

found, and discard it, including the double hyphens, and everything up to the newline or end of file. The parser never sees comments.

Sample Code (scope rules)

```
program ScopeCheck {
    decl globalVar as int;

    routine Outer() returns int {
        decl outerVar as int;        -- 1. Outer's variables

        routine Inner() returns int {
            decl innerVar as int;    -- 2. Inner's variables

            -- Inner's statements
            5 => innerVar;
            yield innerVar;
        }                            -- 3. End of Inner's declaration

        -- Outer's statements
        invoke Inner() => outerVar;
        yield outerVar;
    }                                -- 4. End of Outer's declaration

    -- Module statements
    invoke Outer() => globalVar;
}
```

The Grammar

```
<program>          ::= "program" <identifier> "{" <declaration_list>
<statement_list> "}"

<declaration_list> ::= <var_decl_list> <proc_decl_list>

<var_decl_list>    ::= <var_declaration> <var_decl_list> | ""
<var_declaration>  ::= "decl" <identifier_list> "as" <type> ";"

<proc_decl_list>   ::= <proc_declaration> <proc_decl_list> | ""
<proc_declaration> ::= "routine" <identifier> <formal_params> "returns" <type>
"{" <declaration_list> <statement_list> "}"

<identifier_list>  ::= <identifier> | <identifier> "," <identifier_list>
<type>             ::= "int" | "real" | "bool" | "string"

<formal_params>    ::= "(" <param_list> ")" | "()"
<param_list>       ::= <param> | <param> "," <param_list>
```

```
<param>            ::= <identifier> ":" <type>

<statement_list>   ::= <statement> ";" <statement_list> | <statement> ";"

<statement>        ::= <assignment_stmt>
                     | <conditional_stmt>
                     | <loop_stmt>
                     | <call_stmt>
                     | <return_stmt>
                     | <empty_stmt>

<assignment_stmt>  ::= <expression> "=>" <identifier>

<conditional_stmt> ::= "choose" "[" <expression> "]" "match" "{"
<statement_list> "}" "otherwise" "{" <statement_list> "}"
                     | "choose" "[" <expression> "]" "match" "{"
<statement_list> "}"

<loop_stmt>        ::= "loop" "until" "[" <expression> "]" "do" "{"
<statement_list> "}"

<call_stmt>        ::= "invoke" <identifier> <actual_params>
<actual_params>    ::= "(" <expression_list> ")" | "()"
<expression_list>  ::= <expression> | <expression> "," <expression_list>

<return_stmt>      ::= "yield" <expression>
<empty_stmt>       ::= ""

<expression>       ::= <simple_expr> | <simple_expr> <relop> <simple_expr>
<simple_expr>      ::= <term> | <term> <addop> <simple_expr>
<term>             ::= <factor> | <factor> <mulop> <term>
<factor>           ::= <identifier>
                     | <int_literal>
                     | <real_literal>
                     | <bool_literal>
                     | <string_literal>
                     | "(" <expression> ")"
                     | "not" <factor>

<relop>            ::= "==" | "/=" | "<" | "<=" | ">" | ">="
<addop>            ::= "+" | "-" | "or" | "~"
<mulop>            ::= "*" | "/" | "and"

<bool_literal>     ::= "yes" | "no" | "maybe"
<int_literal>      ::= <digit> <digit_list>
<real_literal>     ::= <digit> <digit_list> "." <digit> <digit_list>
<string_literal>   ::= "`" <character_list> "`"

<identifier>       ::= <letter> <alphanumeric_list>
```

Sample Program

```
program Demo {
    decl x, y as int;
    decl score as real;
    decl msg as string;

    routine calculateBonus(base : real, multiplier : int) returns real {
        decl temp as real;
        base * multiplier => temp;
        yield temp;
    }

    10 => x;
    5.5 => score;
    `Player Score: ` => msg;

    loop until [x > 0] do {
        invoke calculateBonus(score, x) => score;
        x - 1 => x;
    };

    choose [score >= 100.0] match {
        msg ~ `Passed!` => msg;
    } otherwise {
        `Failed!` => msg;
    };
}
```

The Twist

If you take that grammar and feed it to a large language model such as Gemini or ChatGPT, it can probably produce a parser for it at once—it won't be particularly efficient, it certainly won't be elegant, but it will probably work. This is because the process of going from an unambiguous grammar in BNF to a parser is mechanical, *provided* certain assumptions hold. LLM's simply make those assumptions and get on with it. Since the educational value of handing something to ChatGPT and copying the output into Racket is approximately zero, we're going to relax those assumptions. *A naïve parser produced by an LLM will not handle this language correctly.* You will need to modify the LLM's output to make it work. The LLM assumes that the grammar is context-free. We're going to introduce some context.

Context Rules

The primary change is simple: *Keywords are not reserved words*. This means that if a keyword (decl, until, do, etc) appears in a place where that keyword would be an error, then that keyword is just an identifier.

- `decl` is only a keyword if it appears at the beginning of a statement or declaration block; otherwise it's a normal identifier.
  - `decl decl as int;` -- Legal! Declares a variable named 'decl'
  - `decl = 5;` -- Syntax Error (decl is keyword here)
  - `5 => decl;` -- Legal! Assigns 5 to the variable named 'decl'

- The token `match` is only a keyword if the parser is currently inside an active choose statement and has just closed the condition bracket]. In all other contexts (including inside the statement blocks themselves), match is a normal variable identifier.
  - `decl match as bool; yes => match;` -- Legal variable use
  - ```
    choose [match] match {
            no => match;     -- The second 'match' is the keyword;
            };               -- the others are variables!
    ```

- `until` is only a keyword if it immediately follows the keyword `loop`. `do` is only a keyword if it immediately follows the closing bracket `]` of a loop condition.
  - ```
    decl until, do as int;
    10 => until;
    20 => do;
    ```
  - ```
    loop until [until < do] do {
           until + 1 => until;
    };
    ```

- `returns` is only a keyword if the parser is currently processing a routine signature and has just parsed the parameter list closure `)`.
  - ```
    decl returns as string;
      `yes` => returns;
    ```
  - ```
    routine check() returns bool
       { yield yes;
    }
    ```
- The keyword `program` is only a keyword if it appears as the first meaningful (non-comment) token in the file. In any other context, `program` is just an identifier.
    ```

```
    ○ program Demo2 {
          decl program as real;  -- Legal; program is a variable here.
```

Handling Context

To make this work, your compiler cannot use a ‘dumb’ isolated lexer. You have two options, about equally challenging:

- A Stateful Lexer: The parser calls the lexer when it’s ready for the next token. The lexer maintains a communication channel with the parser. When the parser enters a choose statement, it sets a flag expecting_match_keyword = true, which tells the lexer how to classify the next `match` string it sees.
- Parser-Level Demotion: The lexer treats all context-sensitive words as identifiers by default (or as soft-keywords). The parser then explicitly checks both the token type and the string value when matching structure. For example, when looking for the loop boundary, it validates if (current_token.value == "until").

Test String

```
choose [choose == match] match { yes => choose; };
```

Other Functionality

- String comparisons are case-sensitive by default.
- In the event of a syntax error, the parser should return a statement giving the line number: `Syntax error on line 15`. Describing the error, or showing what was encountered, would be helpful but is not required.
    The simplest way to handle this is for the lexer to return the line number as part of each token; this is mostly ignored by the parser, but is there if needed.

Deliverables

You will submit the following:

- Racket (.rkt) file(s) containing your finished, working, documented source code;
- The logs of your LLM interactions, whether saved in a document or as a link to a web page. Include the blind alleys, the badly-worded questions, the irrelevant answers—that’s showing the iterative process. You will not be graded for the quality of your AI prompts, but this is part of the

documentation of the program—here's the code we started with, here's the process of improving the AI, here's where I made the last few fixes myself.

- A 7-10 minute video, similar to the one you did for the first assignment.
    - You must show your code compiling and working correctly for a program without syntax errors, and for a code with syntax errors, showing that your code can handle a syntax error correctly.
    - Discussion of one or more of the following questions, whichever you find most interesting or you feel you have the most to say:
        - Where was AI most helpful? Where was it least helpful?
        - Which parts of the assignment were most challenging for *you*? Most interesting (which isn't necessarily the same thing)?
        - How did you test your code and verify correctness? Give one or more examples of where the AI code didn't work and what *you* did to fix it.
        - Did you notice anything anomalous or unusual about the code itself?
        - Were there any parts of the assignment you didn't think you'd be able to do? How did you find a workaround or get them done?
        - Did the AI suggest Racket functions that don't actually exist? Did you have to reject a solution because it was horribly inefficient?
        - Did using AI help you understand the language or concepts like tail recursion better, or did it make it more confusing?
        - Any other questions about the assignment or process that you found interesting or think worth discussing.
    - Again, your face must be visible throughout the video, and the narration in your voice.

Grading Rubric

Category	Unacceptable (0-59%)	Acceptable (60-89%)	Excellent (90%+)
Functionality (35%)	Fails to handle order of operations. Fails on keywords used as identifiers when it is legal to do so. Scanner crashes on valid input. AST structure is flat or missing branches.	Correctly parses all statements and arithmetic including routine calls. May have minor bugs in complex nested expressions.	Flawless AST generation. Handles edge cases with mathematical precision, including keywords as identifiers and 3-value logic.

Video: Understanding & Logic (25%)	Cannot explain the mapping from BNF rule to Racket function. Confuses lexing and parsing.	Explains how lexer & parser interact, and how keywords as identifiers are handled. Traces through a loop or routine call.	Explains how the grammar hierarchy enforces operator precedence and left-associativity.
Video: Reflection & Analysis (15%)	No mention of formal grammars or ASTs. Treats the project as a simple string-manipulation exercise.	Discusses why recursive descent is easier in Racket than in C/Java	Critiques design of grammar, including keyword/identifier issue. Discusses LL(1) constraints and transition from concrete syntax to abstract represent-ation
Documentation: AI Prompt Logs (15%)	No logs provided, or logs show "blind copying" of logic without any iterative refinement or questioning.	Logs show use of AI for Racket syntax and debugging. Evidence of the student verifying AI-generated logic. Evidence of iterative process of refinement.	Logs demonstrate a "Thought Partner" approach. AI was used to explore grammar alternatives or to refactor code for better readability.
Code Quality & Coding Style (10%)	Variables like x, y, lst used everywhere. No modularity. Code is a single monolithic function.	Follows Racket naming conventions. Functions are clearly mapped to BNF non-terminals. Proper indentation and spacing.	Elegant use of match. Functions are concise and "Self-Documenting." Code reads like a formal specification of the grammar.